VIT®

BHOPAL

PROJECT REPORT

SUBMITTED BY

KARTIKEYA SRIVASTAVA

25BAI10907

COURSE: FUNDAMENTALS IN AI AND ML

2001

TITLE -

Intelligent Disease Prediction System using Bayesian Probabilistic Modelling.

INTRODUCTION –

The "Intelligent Disease Prediction System" is a Python-based diagnostic support tool designed to provide a preliminary assessment of potential health conditions based on a user's symptoms. By analysing a historical dataset of diseases and their corresponding symptom patterns, the system calculates the probability of various diseases and presents the top matches. It serves as an accessible entry point for health monitoring.

MOTIVATION OF THE PROJECT –

In many regions, immediate access to medical professionals is limited, and people often resort to searching symptoms online, which can lead to misinformation. The motivation behind this project is to create a structured, data-driven approach to symptom analysis that uses mathematical probability rather than simple keyword matching, providing a more reliable "first-check" for common illnesses.

PROBLEM STATEMENT-

Users often experience common symptoms (fever, cough, fatigue) that overlap across multiple diseases. Manually identifying whether these symptoms point to a common cold, influenza, or something more serious like malaria is difficult. There is a need for a system that can quantify the likelihood of multiple diseases simultaneously based on a weighted analysis of symptom frequency.

OBJECTIVE OF THE THE PROJECT-

- To develop a system that reads symptom data from a CSV dataset.
- To implement a probabilistic algorithm (Simplified Naive Bayes) to calculate disease scores.
- To provide a user-friendly Command Line Interface (CLI) for symptom input.
- To output the top three most likely diseases with a "match percentage."

EXISTING METHODS-

- Manual Diagnosis: Visiting a clinic for physical examination and lab tests.
- Rule-Based Expert Systems: Systems that use "If-Then" logic (e.g., If Fever AND Cough, then Flu).
- Online Health Databases: Static websites where users read through lists of symptoms.

PROS AND CONS OF THE STATED METHODS-

Method	**Pros**	**Cons**
Manual Diagnosis	Most accurate, provides treatment.	Time-consuming, expensive, not always accessible.
Rule-Based Systems	Easy to implement for simple cases.	Too rigid; fails when symptoms are missing or atypical.
Online Databases	Huge amount of information.	Overwhelming for users; can cause unnecessary anxiety.
Current System	Probabilistic, handles missing data well.	Limited by the quality and size of the dataset.

HARDWARE AND SOFTWARE REQUIREMENTS-

Hardware:

- Processor: Dual-core 2.0 GHz or higher.
- RAM: 4 GB minimum.
- Storage: 50 MB (minimal footprint).

Software:

- Operating System: Windows, macOS, or Linux.
- Programming Language: Python 3.8+.
- Data Format: CSV (Comma Separated Values).

METHODOLOGY AND GOAL-

The system follows a Data-Driven Inferential Methodology:

1. Data Acquisition: Load disease-symptom mappings from disease_dataset.csv.
2. Frequency Analysis: Calculate how often each symptom occurs for each disease.

3. Smoothing: Apply Laplace Smoothing to ensure no probability is zero.
4. Inference: Compare user inputs against the learned patterns.
5. Goal: To provide a ranked list of likely conditions to help users decide the urgency of medical attention.

FUNCTIONAL MODULES DESIGN AND ANALYSIS-

- Data Loading Module (load_data): Parses the CSV file into a list of dictionaries.
- Training Module (train): Iterates through the data to build dictionaries for disease frequency and symptom frequency.
- Input Module (get_user_symptoms): An interactive loop that asks the user about each symptom in the dataset.
- Prediction Module (predict): The core engine that calculates scores using the product of conditional probabilities.
- Display Module (show_results): Formats and prints the final output.

ALGORITHM DEVELOPMENT-

The system utilizes a **Simplified Naive Bayes Classifier** logic:

- **Prior Probability:** $P(\text{Disease}) = \frac{\text{Count of Disease Records}}{\text{Total Records}}$
- **Conditional Probability:** $P(\text{Symptom}|\text{Disease}) = \frac{\text{Symptom Count for Disease}+1}{\text{Disease Count}+2}$
- **Total Score:** For a given set of user symptoms S, the score for a disease D is:

$$\text{Score}(D) = P(D) \times \prod_{i=1}^{n} P(s_i|D)$$

where s_i is 1 if the user has the symptom and 0 otherwise.

CODING-

The core prediction logic is implemented as follows:

```
def predict(user_symptoms, disease_count,
symptom_count, symptoms_list):
    scores = {}
    total_records = sum(disease_count.values())


    for disease in disease_count:
        # Start with the prior probability
        score = disease_count[disease] / total_records
        count = disease_count[disease]
```

```
        for symptom in symptoms_list:
            # Probability calculation with Laplace Smoothing
            prob = (symptom_count[disease][symptom] + 1)
/ (count + 2)


            # If user has the symptom, multiply by prob; if
not, multiply by (1-prob)
            if user_symptoms[symptom] == 1:
                score *= prob
            else:
                score *= (1 - prob)
        scores[disease] = score
    return sorted(scores.items(), key=lambda x: x[1],
reverse=True)
```

OUTPUT-

Dataset loaded: 50 records found

Diseases in dataset: ['Common Cold', 'Influenza', 'COVID-19', 'Malaria', ...]

Do you have fever? (y/n): y

Do you have cough? (y/n): y

...

--- Prediction Results ---

#1 Influenza -> 78.4% match

#2 COVID-19 -> 15.2% match

#3 Common Cold -> 6.4% match

NOTE: This is not a medical diagnosis. Please consult a doctor.

KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM-

- Dynamic Feature Detection: The system automatically detects symptoms based on CSV headers, making it easy to add new symptoms without changing the code.
- Laplace Smoothing: Prevents "Zero Probability" errors where a single missing symptom would result in a 0% match.
- Relative Confidence Ranking: Shows the "Top 3" to account for the overlapping nature of symptoms.

SIGNIFICANT PROJECT OUTCOMES-

- Successfully categorized 10 distinct diseases (e.g., Malaria, COVID-19, Typhoid).
- Tracks 20 unique symptoms, from basic (fever, cough) to specific (rash, joint pain).
- Achieved a logical ranking system where similar diseases appear together in the results.

TESTING AND REFINEMENT-

- Unit Testing: Verified load_data returns the correct dictionary structure.
- Logic Testing: Inputted symptoms specifically matching "Dengue" (fever, rash, joint pain) to ensure it ranks #1.
- Refinement: Added .strip().lower() to user inputs to handle "Y" or "y" gracefully.

PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS-

- Telemedicine Triage: Can be used as a pre-screening bot for healthcare apps.
- Remote Health Kits: Useful in rural areas with low bandwidth where a simple CLI tool is easier to run than a heavy web app.
- Educational Tools: Helps medical students understand the statistical overlap between different syndromes.

CONTRIBUTION/ FINDINGS OF THE PROJECT-

- Finding: Symptoms like "fever" are highly non-specific and appear in almost all diseases in the dataset, whereas "rash" is highly specific to Dengue and Chickenpox.
- Contribution: Developed a lightweight, zero-dependency prediction engine that runs on any machine with Python.

LIMITATION/CONSTRAINTS OF THE SYSTEM-

- Binary Input: Cannot distinguish between "high fever" and "mild fever."
- Small Dataset: With only 50 records, the statistical power is limited.
- No Severity Check: Does not ask for the duration of symptoms.

CONCLUSIONS-

The Intelligent Disease Prediction System successfully demonstrates that probabilistic logic can effectively rank potential health conditions based on binary symptom inputs. While it is not a substitute for clinical diagnosis, it provides a structured and logical way to interpret complex symptom patterns.

FUTURE ENHANCEMENTS-

- Fuzzy Logic: Allowing users to input "Mild," "Moderate," or "Severe" symptoms.
- Graphical User Interface (GUI): Developing a web or mobile interface using Flask or Streamlit.
- API Integration: Connecting to real-time disease outbreak data (like CDC feeds).

REFERENCES-

- Naive Bayes Classifiers - Wikipedia / Machine Learning Documentation.
- Python csv Module Documentation - docs.python.org.
- Laplace Smoothing Techniques - Introduction to Probability and Statistics.